Apellidos: ..... Nombre: ..... Grup: .....

Examen EP2 (*Temas 8, 9, 10, 11, 12, 13 y 14)*

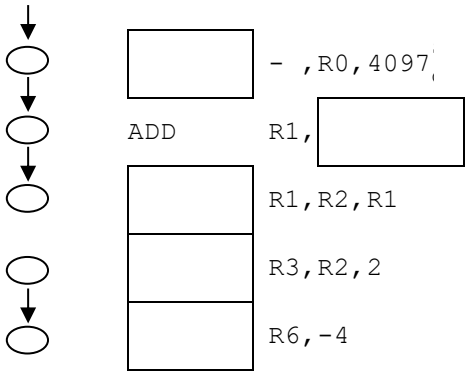
- Duración del examen: 2 horas y 30 minutos.
- La solución se tiene que escribir en el espacio reservado para ello en el propio enunciado.
- No podéis utilizar calculadora, móvil, apuntes, etc.
- La solución y las notas se publicarán en Atenea el martes 4 a lo largo de la tarde. La revisión será el jueves 6 a las 10:00.

Ejercicio 1 *(1 puntos)*

Dado el siguiente fragmento de código en C (el código no tiene que hacer algo útil), indicad cómo se implementaría en un procesador que use la UPG del procesador **Harvard uniciclo**. Primero utilizando una Unidad de Control de **propósito específico** (UCe) y luego utilizando una unidad de control de **propósito general** (UCG). Todos los datos son **naturales**.

```
if (R0 > 4097) {
      R1= R2-2*R3;
} else {
      R3=R2+2;
}
R6=-4;
```

a) Completad el fragmento del grafo de estados de la UC de **propósito específico** para que junto con la UPG del procesador **Harvard uniciclo** formen un procesador que realice la funcionalidad descrita en los fragmentos de código anteriores. Indicad los arcos que faltan, las etiquetas de los arcos (z, !z, o nada) y completad las casillas de cada palabra de control que se especifica con mnemotécnicos a la derecha de cada nodo del grafo. *(0.5 puntos)*



b) Completad el fragmento de programa en lenguaje ensamblador SISA para realice la funcionalidad descrita en el fragmento de código en C (el código no tiene que hacer algo útil). El código SISA ya escrito siempre utiliza el registro `R7` para valores temporales. En las comparaciones, hay que interpretar los datos como valores **naturales**. Rellenad la parte subrayada que falta. No se pueden usar etiquetas. *(0.5 puntos)*

@Mem	
0x0000	_____ R7, ____
0x0002	MOV___ R7, ____
0x0004	CMP___ R7, R0, R7
0x0006	B____ R7, ____
0x0008	ADD R1, _____
0x000A	_____ R1, R2, R1
0x000C	B____ R7, ____
0x000E	_____ R3, R2, ____
0x0010	_____ R6, -4

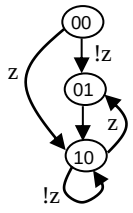
Ejercicio 2 *(0.9 puntos)*

Completa la siguiente tabla ensamblando las instrucciones en ensamblador SISA o desensamblando las instrucciones en lenguaje máquina según sea necesario. Indica poniendo **NA** en la casilla aquellos casos en los que la instrucción no corresponda al lenguaje SISA.

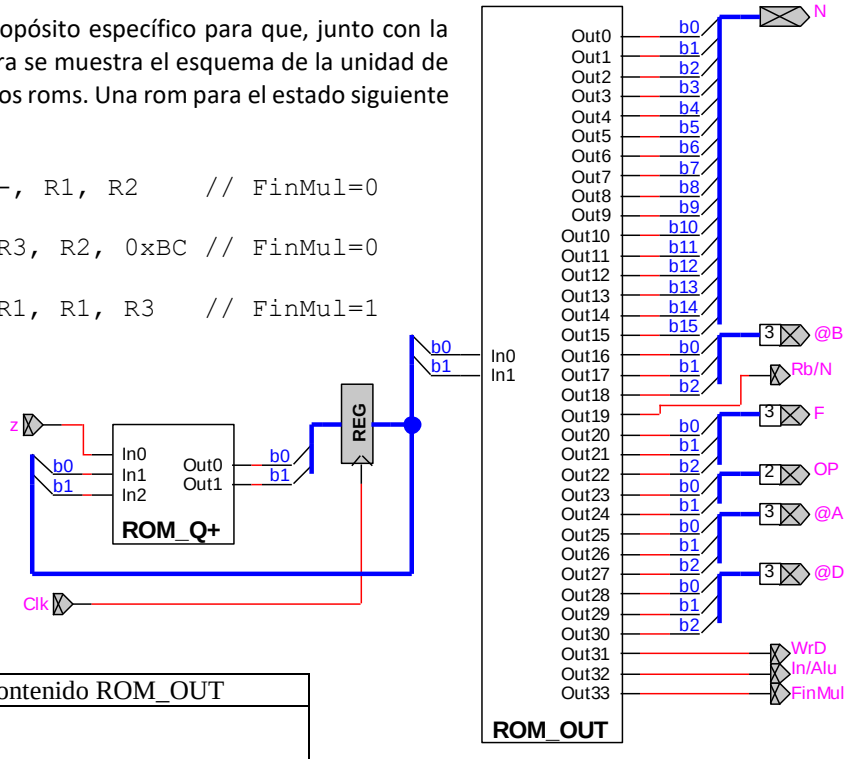
Lenguaje máquina SISA	Lenguaje ensamblador SISA
`0x0623`	
	`IN R3, 0x66`
`0x2F3A`	

Ejercicio 3 *(0.8 puntos)*

Dado el siguiente grafo de estados de una UC de propósito específico para que, junto con la UPG básica, formen un PPE que lo ejecute. En la figura se muestra el esquema de la unidad de control correspondiente a una implementación con dos roms. Una rom para el estado siguiente (ROM_Q+) y otra rom para las salidas (ROM_OUT).




```
CMPEQ -, R1, R2    // FinMul=0
ADDI  R3, R2, 0xBC // FinMul=0
SHL   R1, R1, R3   // FinMul=1
```



Indica el contenido de las ROMS en hexadecimal. Utilizad el valor 0 para los bits que sean x. Si los contenidos no están en hexadecimal no se corregirá el ejercicio.

@ROM	Contenido ROM_Q+
@0	
@1	
@2	
@3	
@4	
@5	
@6	
@7	

@ROM	Contenido ROM_OUT
@0	
@1	
@2	
@3	

Ejercicio 4 *(0.9 puntos)*

Indicad qué cambios se producen en el estado del computador SISC Von Neumann después de ejecutar cada una de las instrucciones de la tabla suponiendo que antes de ejecutarse cada una de ellas el PC vale `0x26AC`, el contenido de todos los registros pares es `0x6789` y el de los registros impares es `0x0123`, el byte contenido en todas las direcciones pares de la memoria es `0x64` y el de todas las impares es `0x32` y el contenido de todos los puertos de entrada es `1` y el de los de salida es `2`. Utilizad la notación MEM_b[0x...]=0x... y/o MEM_w[0x...]=0x... para indicar cualquier cambio en la memoria y PORTIN[0x...]=0x... para los puertos de entrada y PORTOUT[0x...]=0x... para los de salida.

Instrucción a ejecutar	Cambios en el estado del computador
`STB 0x12(R2), R4`	
`BNZ R0, -6`	
`MOVHI R3, 0x0D`	

Apellidos: …… Nombre: …… Grup: ……

Ejercicio 5 *(2.2 puntos)*

El programa en ensamblador SISA que se muestra a continuación se ha traducido a lenguaje máquina situando la sección de datos (.data) a partir de la dirección `0x0800` y la sección de código (.text) a partir de la dirección `0xA000`.

```
.data
        A: .word 0x000F, 0xAB00, 0x0100, 0xFFFE, 0x1234
        B: .word 0xBEEF, 0xDEAD, 0xCAFE, 0xBABE, 0xACDC
           .word 0
.text
        MOVI  R1, lo(A)
        MOVHI R1, hi(A)
        MOVI  R2, lo(B)
        MOVHI R2, hi(B)
        SUB   R3, R2, R1
        MOVI  R7, 15
        MOVI  R0, 0
loop:   LD    R4, 0(R1)
        LD    R5, 0(R2)
        XOR   R6, R4, R5
        AND   R6, R6, R7
        BNZ   R6, et2
et1:    ADDI  R0, R0, 1
        BZ    R6, cont
et2:    ADDI  R0, R0, -1
cont:   ADDI  R1, R1, 2
        ADDI  R2, R2, 2
        ADDI  R3, R3, -2
        BNZ   R3, loop
        ST    0(R2), R0
.end
```

Una vez ensamblado y cargado el programa en la memoria y antes de comenzar su ejecución:

a) ¿A qué direcciones de memoria corresponden las etiquetas o direcciones simbólicas siguientes? *(0.4 puntos)*

B=0x et1=0x

b) ¿Cuál es la dirección de memoria y su contenido donde han quedado almacenadas las siguientes instrucciones una vez cargado el programa? *(0.6 puntos)*

Instrucción	@ memoria y contenido
`MOVHI R1, hi(A)`	Mem_w[0x] = 0x
`LD R4, 0(R1)`	Mem_w[0x] = 0x
`BZ R6, cont`	Mem_w[0x] = 0x

c) Una vez ejecutado el programa en el SISC von Neumann, ¿a qué dirección de memoria y qué valor está escribiendo la instrucción ST?. *(0.5 puntos)*

Dirección=0x Valor= 0x

d) Si se ejecuta el código mostrado, ¿Cuántas instrucciones se ejecutan, cuántas son lentas y cuántas son rápidas en la Harvard multiciclo? Si se ejecuta el código en las tres versiones de los computadores SISC (Harvard uniciclo, el Harvard multiciclo y el Von Neumann) ¿Cuántos ciclos tarda en ejecutarse el código en cada uno de los tres computadores? *(0.7 puntos)*

Nº total de instruc. ejecutadas = Nº de instruc. rápidas = Nº de instruc. Lentas =

Nº de ciclos (H. uniciclo) = Nº de ciclos (H. multiciclo) = Nº de ciclos (Von Neumann) =

Ejercicio 6 *(1 punto)*

Cada uno de los apartados pregunta sobre un ciclo concreto de la ejecución de varias instrucciones en el SISC Von Neumann. Escribid el contenido del registro IR, el contenido de la ROM_OUT y el valor de los bits de la **palabra de control** que genera el bloque **SISC CONTROL UNIT** durante el ciclo a que hace referencia cada apartado**.** Para cada apartado/fila se indica el nodo/estado de la UC en ese ciclo y la instrucción (en ensamblador) que está almacenada en el IR en ese ciclo. Podéis ver el grafo de estados de Moore de la UC en el anexo. Suponed que el contenido de todos los registros, `Rk` para k=0,...7, antes de ejecutarse cada instrucción es 0.

a) Indica el contenido de la ROM_OUT en hexadecimal usando las conexiones en el orden que están en el anexo. Utilizad el valor 0 para los bits que sean x (solo para este apartado). *(0.5 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Contenido ROM_OUT** (en hexadecimal)
Jalr	`JALR R1,R2`	`0x`
Addr	`ST -1(R4),R5`	`0x`

b) Indica el valor de los bits de la **palabra de control** que genera el bloque **SISC CONTROL UNIT** durante el ciclo a que hace referencia cada apartado**. Poned x siempre que no se pueda saber el valor de un bit** (ya que no podemos suponer cómo se han implementado las x en la ROM_OUT) **o sea irrelevant**e para el estado en cuestión. *(0.5 puntos)*

Apartado	**Nodo / Estado** (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Palabra de Control**																		
			@A	@B	Pc/Rx	Ry/N	OP	F	P/I/L/A	@D	WrD	Wr-Out	Rd-In	Wr-Mem	LdIr	LdPc	Byte	Alu/R@	R@/Pc	N (hexa)	ADDR-IO (hexa)
a	Jalr	`JALR R1,R2`																			
b	Addr	`ST -1(R4),R5`																			

Ejercicio 7 *(1.2 puntos)*

Especificad el **camino crítico** (indicando la suma ordenada de los tiempos de propagación de los bloques por los que pasa) y calculad el **tiempo de ciclo mínimo** para que el computador **SISC Von Neumann** pueda ejecutar correctamente el tipo de instrucción SISA que se indica en cada apartado (este sería el tiempo de ciclo mínimo del computador si solo ejecutara instrucciones como la indicada u otras que requieran menor tiempo). No tenéis que añadir ningún porcentaje de seguridad en el cálculo del tiempo de ciclo mínimo. Suponed que los tiempos de propagación de los bloques que forman el computador son los siguientes:

T_p(ROM_Q+) = 60 u.t.

T_p(ROM_OUT) = 80 u.t.

T_p(MUX-2-1) = 50 u.t.

T_p(MUX-4-1) = 100 u.t.

T_p(REG) = 100 u.t. // Tiempo de propagación de un registro.

T_p(REGFILE) = 200 u.t. // Tiempo de lectura del banco de registros

T_p(ALU-slow) = 800 u.t. // Tp de la ALU para las operaciones/funciones lentas: ADD, SUB, CMP*.

T_p(ALU-quick) = 400 u.t. // Tp de la ALU para las operaciones/funciones rápidas: cualquier otra distinta de ADD, SUB, CMP*.

T_{acc}(MEMORY) = 1000 u.t. // Tiempo de acceso (para la lectura o escritura) a la memoria

T_p(AND-2) = T_p(OR-2) = 20 u.t.

T_p(NOT) = 10 u.t.

El tiempo de propagación de un bloque combinacional (T_p) y el tiempo de acceso a memoria para realizar una lectura (T_{acc}) es el tiempo desde que están estables todas las entradas necesarias hasta que se estabilizan las salidas requeridas al valor correcto para las entradas aplicadas. Desconocemos como se han implementado internamente los bloques (y podría ser de forma diferente a los vistos en clase). Recordad que un registro con señal de carga (Ld), REGwLd, está construido con un REG y un MUX-2-1 (no os damos el esquema interno del REGwLd, porque lo tenéis que saber).

a) T_c correspondiente al nodo de **D** (decode).

b) T_c correspondiente al nodo de **Stb**.

c) T_c correspondiente al nodo de **Movi**.

Apellidos: ..... Nombre: ..... Grup: .....

Ejercicio 8 *(2 puntos)*

Completad el diseño del SISC Von Neumann para que pueda ejecutar, además de las 25 instrucciones originales SISA, una nueva instrucción que almacene un word en memoria y además incremente en 2 el registro base (Ra), que tiene el formato y codificación, la sintaxis ensamblador y la semántica siguientes:

Codificación: `1011 aaa bbb nnnnnn`
Sintaxis: `ST2 N6(Ra), Rb`
Semántica: `PC=PC+2; Mem`$_w$`[(Ra + SE(N6))&~1]=Rb; Ra=Ra+2;`

Sin modificar el hardware y sólo modificando el contenido de las ROM's, completad el diseño del computador para que ejecute, además de las instrucciones originales, la nueva instrucción ST2 en, como máximo, 6 ciclos.

a) Completad el contenido de la siguiente tabla que indica, mediante una fila para cada nodo del grafo de estados de la unidad de control, la acción (o acciones en paralelo) que se realiza en el computador en cada uno de los, **como máximo**, 6 ciclos/nodos que requiere la ejecución de la nueva instrucción (Fetch, Decode, y los, **como máximo**, cuatro ciclos/nodos de la ejecución propiamente dicha. Para especificar las acciones usad el mismo lenguaje de transferencia de registros que en la documentación (con el que ya hemos completado las acciones de los nodos E0/F y E1/D). *(1 punto)*

Nodo	Mnemotécnico	Acciones
E0	F	IR ← Mem$_w$[PC] // PC ← PC + 2
E1	D	RX ← Ra // RY ← Rb // R@ ← PC + SE(N8)*2

Si la solución propuesta en el apartado a) no permite ejecutar correctamente la nueva instrucción, no se corregirán los apartados b) ni c).

b) Completad (poniendo 0, 1 o x en cada bit) las, **como máximo**, cuatro filas de la tabla que especifican el contenido de la ROM_OUT para los nodos posteriores a Decode. **Poned x siempre que el valor de un bit no importe**. *(0,6 puntos)*

@ROM	Bnz	Bz	WrMem	RdIn	WrOut	WrD	LdIr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	P/I/L/A$_1$ P/I/L/A$_0$	OP$_1$ OP$_0$	MxN$_1$ MxN$_0$	MxF	F$_2$ F$_1$ F$_0$	Mx@D$_1$ Mx@D$_0$

c) Completad la dirección o contenido, según corresponda, de la ROM_Q+ del SISC Von Neumann con esta instrucción. *(0.4 puntos)*

`ROM_Q+[0x0B6]=0x` `ROM_Q+[0x ]=0x10`